

day03 课堂笔记

day03 课堂笔记

0. 复习和反馈

1. 异常的概念[了解]

2. 捕获单个异常[掌握]

3. 捕获多个异常[掌握]

4. 打印异常信息[掌握]

5. 捕获所有的异常[掌握]

6. 异常的完整结构[掌握]

7. 异常的传递[理解]

8. 抛出自定义异常[理解]

模块

模块的制作

模块的导入

模块中的变量 `__all__`

模块中的变量 `__name__`

模块的注意事项

包

学生管理系统-面向对象版

面向对象开发的一般方式

搭建框架代码

实现添加学生的功能
删除/修改/查询 学生
保存
读取

0. 复习和反馈

```
1  实例属性：
2      类外部： 实例对象.属性名 = 属性值
3      类内部： self.属性 = 属性值  (__init__)
4
5  类属性：
6      类内部，方法外部直接定义的变量，就是类属性
7
8  实例方法：
9      直接定义的方法
10     def 方法名(self):
11         # 方法使用了实例属性
12         self.属性
13         pass
14  类方法：
15     @classmethod
16     def 方法名(cls):
17         类名.类属性
18         cls.类属性
```

```
19         pass
20 静态方法:
21     @staticmethod
22     def 方法名():
23         pass
```

1. 异常的概念[了解]

- 1 异常的组成:
- 2 异常类型: 异常具体的描述信息

```
1  # 异常: 程序运行过程中, 代码遇到错误, 给出错误的提示
2
3  print('其他的代码.....')
4  num = input('请输入一个数字:')
5  # ZeroDivisionError: division by zero
6  # ValueError: invalid literal for int() with base 10: 'a'
7  num = 10 / int(num)
8  print('计算得到的结果是:', num)
9  print('其他的代码.....')
10
```

2. 捕获单个异常[掌握]

```

1  # 异常：程序代码在运行过程中遇到的错误，程序会报错，会终止程序代码的运行。
2  # 异常捕获：是指在程序代码运行过程中，遇到错误，不让程序代码终止，让其继续运行，同时可以给使用者一个提示信息
3  # 并记录这个错误，便于后期改进
4  """
5  try:
6      可能发生异常的代码
7  except 异常的类型:
8      发生异常执行的代码
9  """
10
11
12  print('其他的代码.....')
13  num = input('请输入一个数字:')
14  # ZeroDivisionError: division by zero
15  # ValueError: invalid literal for int() with base 10: 'a'
16  try:
17      num = 10 / int(num)
18      print('计算得到的结果是:', num)
19  except ZeroDivisionError:
20      print('你输入有误,请再次输入')
21
22  print('其他的代码.....')

```

3. 捕获多个异常[掌握]

```

1  """
2  try:
3      可能发生异常的代码
4  except (异常的类型1, 异常类型2, ...):
5      发生异常执行的代码
6
7
8  """
9
10
11  print('其他的代码.....')
12  num = input('请输入一个数字:')
13  # ZeroDivisionError: division by zero
14  # ValueError: invalid literal for int() with base 10: 'a'
15  try:
16      a = int(num)
17      num = 10 / a
18      print('计算得到的结果是:', num)
19  except (ZeroDivisionError, ValueError):
20      print('你输入有误,请再次输入')
21
22  print('其他的代码.....')

```



```

22 """
23 try:
24     可能发生异常的代码
25 except 异常类型1:
26     发生异常1,执行的代码
27 except 异常类型2:
28     发生异常2,执行的代码
29 except ...:
30     pass
31 """
32 print('其他的代码.....')
33 num = input('请输入一个数字:')
34 # ZeroDivisionError: division by zero
35 # ValueError: invalid literal for int() with base 10: 'a'
36 try:
37     a = int(num)
38     num = 10 / a
39     print('计算得到的结果是:', num)
40 except ZeroDivisionError:
41     print('你输入有误,请再次输入')
42 except ValueError:
43     print('输入有误,请输入数字')
44
45 print('其他的代码.....')

```

C:\develop\python35\python.exe
其他的代码.....
请输入一个数字:abc
输入有误,请输入数字
其他的代码.....

激汗 Windows

4. 打印异常信息[掌握]

```

1 """
2 try:
3     可能发生异常的代码
4 except (异常的类型1, 异常类型2, ...) as 变量名:
5     发生异常执行的代码
6     print(变量名)
7 """
8
9
10 print('其他的代码.....')
11 num = input('请输入一个数字:')
12 # ZeroDivisionError: division by zero
13 # ValueError: invalid literal for int() with base 10: 'a'
14 try:
15     a = int(num)
16     num = 10 / a
17     print('计算得到的结果是:', num)
18 except (ZeroDivisionError, ValueError) as e:
19     print('你输入有误,请再次输入', e)
20
21 print('其他的代码.....')

```

5. 捕获所有的异常[掌握]

```
1  try:
2      可能发生异常的代码
3  except:    # 缺点, 不能获取异常的描述信息
4      发生异常执行的代码
5
6
7  =====
8  try:
9      可能发生异常的代码
10 except Exception as e:
11     发生异常执行的代码
12     print(e)
13     pass
14
15 # Exception 是常见异常类的父类,
16 ZeroDivisionError --> ArithmeticError -->
    Exception --> BaseException ----> object
17 ValueError --> Exception --> BaseException
    ----> object
```

```
1 print('其他的代码.....')
2 num = input('请输入一个数字:')
3 try:
4     a = int(num)
5     num = 10 / a
6     print('计算得到的结果是:', num)
7     f = open('1.txt', 'r')
8 except Exception as e:
9     print('你输入有误,请再次输入', e)
10
11 print('其他的代码.....')
```

6. 异常的完整结构[掌握]

```
1 try:
2     可能发生异常的代码
3 except Exception as e:
4     发生异常执行的代码
5     print(e)
6 else:
7     代码没有发生异常,会执行
8 finally:
9     不管有没有发生异常,都会执行
```

```
1 print('其他的代码.....')
2 num = input('请输入一个数字:')
3 try:
4     a = int(num)
5     num = 10 / a
6     print('计算得到的结果是:', num)
7 except Exception as e:
8     print('你输入有误,请再次输入', e)
9 else:
10    print('没有发生异常,我会执行')
11 finally:
12    print('不管有没有发生异常,我都会执行')
13
14 print('其他的代码.....')
```

7. 异常的传递[理解]

- 1 是 python 异常处理的底层机制,是原理层面的,不需要我们自己写代码实现,是 python 已经实现好的.
- 2
- 3 异常传递: 当一行代码发生异常之后,会向外层将这个异常进行传递,指导被捕获或者程序报错为止.
- 4 1. try 嵌套
- 5 2. 函数嵌套

```

1 print('其他的功能代码.....')
2 num = input('请输入数字:')
3 try:
4     try:
5         a = int(num) # ValueError
6     except ZeroDivisionError:
7         print('发生异常')
8     finally:
9         print('我都执行了....')
10
11     num = 10 / a
12     print(f'计算的结果<<{num}>>')
13 except Exception as e:
14     print(e)
15
16
17 print('其他的功能代码.....')

```

内层 try 代码中发生的异常，没有捕获，会向外层进行传递

其他的功能代码.....
 请输入数字:>? qwer
 我都执行了....
 invalid literal for int() with base 10: 'qwer'

```

1 def func1():
2     print('-----1-----')
3     num = input('请输入数字') # 0
4     num = 10 / int(num) # 假设0 是计算出来的
5     print(num)
6     print('-----2-----')
7
8
9 def func2():
10    print('-----3-----')
11    func1()
12    print('-----4-----')
13
14
15 try:
16    print('-----5-----')
17    func2()
18    print('-----6-----')
19 except Exception as e:
20    print('-----7-----')
21    print(e)
22
23 # 5 3 1 7

```

8. 抛出自定义异常[理解]

```
1  程序代码为什么会报错?
2  因为不符合语法, 因为 python 的作者在代码中使用了
   if 判断, 如果除数为 0 , 就会在代码中抛出异常错误,
3  抛出异常:
4      raise 异常对象  # 当程序代码遇到 raise 的时
   候, 程序就报错了
5
6  异常对象 = 异常类(参数)
7
8  抛出自定义异常:
9      1. 自定义异常类, 继承 Exception 或者
   BaseException
10     2. 选择书写, 定义 __init__ 方法, 定义 __str__ 方
   法
11     3. 在合适的时机抛出异常对象即可
```

```

1  # 1. 定义异常类, 密码长度不足的异常
2  class PasswordLengthError(Exception):
3      pass
4      # 继承父类的方法
5
6  def get_password(): # 等同于系统定义函数
7      password = input('请输入密码:')
8      if len(password) >= 8:
9          print('密码长度合格')
10     else:
11         # 抛出异常
12         raise PasswordLengthError('密码长度不足8位')
13         # print('密码长度不足8位')
14
15     try:
16         get_password() # 调用系统的函数
17     except PasswordLengthError as e:
18         print(e) # 异常类的 __str__ 方法
19
20     print('其他代码.....')
21

```

模块

```

1  import random
2  random.randint(a, b) 产生[a, b] 之间的随机整数,
   包含 b的
3  import os
4
5  python 就是一个 python 代码文件, 是别人已经写好的代
   码文件, 文件中的函数类以及变量, 我们都可以使用。
6
7  使用模块的好处: 直接使用别人已经实现好的功能,

```

模块的制作

- 1 模块就是一个 python 文件
- 2 制作一个模块, 其实就是定义一个 python 代码文件
- 3 注意点: 模块的名字要遵循标识符的规则(由字母, 数字和下划线组成, 不能以数字开头)
- 4
- 5 模块中可以定义变量, 定义函数, 定义类

模块的导入

- 1 想要使用模块中的内容, 必须先导入模块
- 2 注意点: 如果导入的是自己书写的模块, 使用的模块和代码文件需要在一个目录中

```
1 # 想要使用模块中的内容, 必须先导入模块
2 # 方法一 import 模块名
3 # 使用: 模块名.功能名
4 import my_module1
5
6 print(my_module1.num) # 使用my_module1中的变量num
7 my_module1.func() # 调用my_module1中 func函数
8 dog = my_module1.Dog() # 调用my_module1中的类创建对象
9 dog.show_info()
```

```
10
11 # 方法二 from 模块名 import 功能名1, 功能名2, ....
12 # 使用: 功能名
13 # 注意点: 如果存在同名的方法名, 则会被覆盖
14 from my_module2 import func, num
15 from my_module1 import num
16 func()
17 print(num)
```



```

18
19 # 方法三 from 模块名 import * # 将模块中所有的功能进行导入
20 # 使用: 功能名
21 from my_module2 import *
22
23 print(num)
24 func()
25 dog = Dog()
26 dog.show_info()

```

```

27
28 # as 起别名. 可以对模块和功能起别名,
29 # 注意: 如果使用as别名, 就不能再使用原来的名字
30 import my_module1 as mm1
31 from my_module1 import func as m1_func
32 from my_module2 import func as m2_func
33 mm1.func()
34 m1_func()
35 m2_func()

```

模块中的变量 `__all__`

```

1 """
2 __all__ 变量, 可以在每个代码文件中(模块中)定义, 类型是元组, 列表
3 作用: 影响 from 模块名 import * 导入行为, 另外两种导入行为不受影响
4 1. 如果没有定义__all__ 变量, 模块中的所有功能, 都可以被导入
5 2. 如果定义__all__ 变量, 只能导入 变量中定义的内容
6 """
7 from my_module3 import *
8
9 print(num)
10 func()
11 dog = Dog()
12 dog.show_info()
13
14 __all__ = ['num', 'func']
15 num = 3
16
17 def func():
18     print('my_module3 func ...')
19
20 class Dog(object):
21     @staticmethod
22     def show_info():
23         print('这是一个Dog类 , r')

```

```

Run: 11-__all__变量 x
C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day10/11-__all__变量.py
Traceback (most recent call last):
  File "C:/Users/nl/Desktop/43/day10/11-__all__变量.py", line 11, in <module>
    dog = Dog()
NameError: name 'Dog' is not defined
3
my_module3 func .....
Process finished with exit code 1

```

模块中的变量 `__name__`

12-__name__变量.py ×

```
1 import my_calc # 模块的导入, 会执行模块中的代码
2
3 my_calc.add(100, 200)
4
```

程序代码的运行:
1. 直接运行代码文件
2. 将代码文件作为模块, 导入运行

my_calc.py ×

```
1 def add(a, b):
2     print(a + b)
3
4
5 if __name__ == '__main__':
6     add(10, 20)
7     # __name__ 变量, 在每个模块即代码文件中都有, 是系统自己定义的
8     # 1. 直接运行当前代码, 值为 __main__
9     # 2. 把文件作为模块导入时, 运行, 结果是 my_calc(文件名)
10    print(__name__)
11
12
```

判断主模块 (直接运行的代码)

10 main if __name__ == '__main__'

11 Press Ctrl+, to choose the selected (or first) suggestion and insert a dot afterwards Next Tip

12 main

13

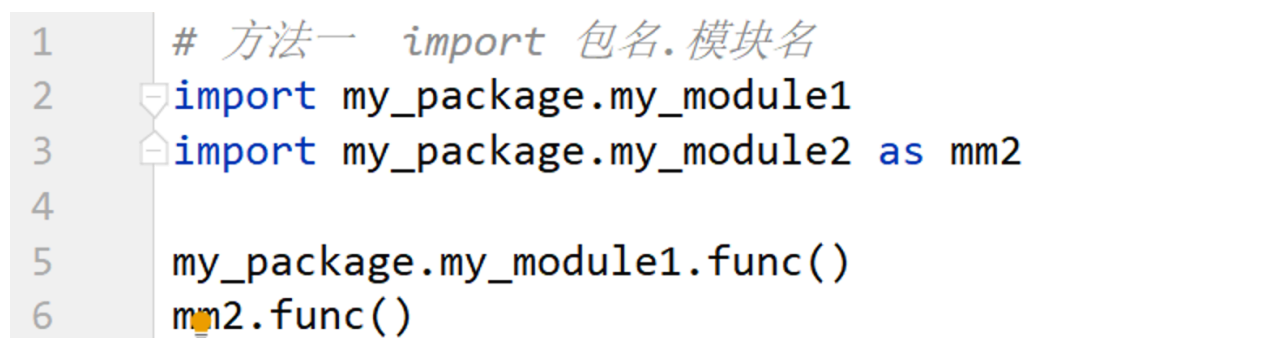
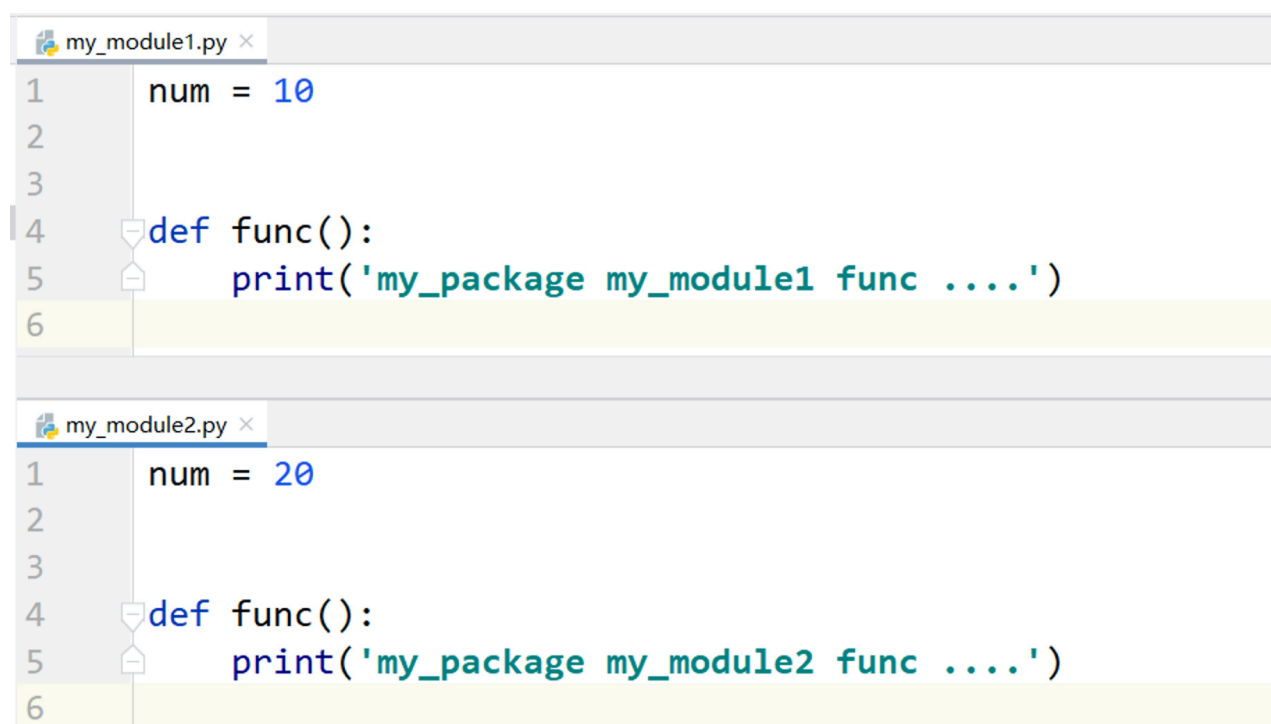
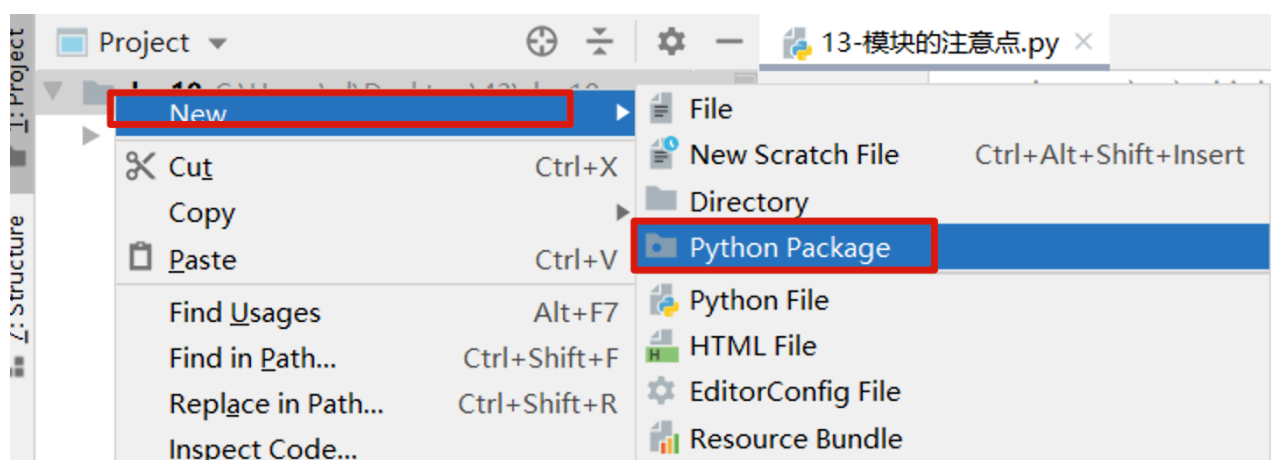
直接回车, 快速的生成

模块的注意点

```
1 # 自己定义的模块名字, 不要和系统中你要使用的模块名字相同
2 import random
3 import sys
4
5 # 模块的搜索顺序, 当前目录 ---> 系统目录 ---> 程序报错
6 print(sys.path)
7
8
9 a = random.randint(1, 5)
10 print(a)
```

包

- 1 包：功能相近或者相似的模块放在一个目录中,并在目录中定义一个 `__init__.py` 文件,这个目录就是包



```
8 # 方法2 from 包名.模块名 import 功能名
9 from my_package.my_module1 import func
10 from my_package.my_module2 import *
11 func()
12
```

```
13 # 方法三 from 包名 import * # 导入的是__init__.py中的内容
14 from my_package import *
15 func()
```

学生管理系统-面向对象版

```
1 面向对象： 编程思想。 偷懒
2
3 对象 = 管理系统类()
4 对象.start()
5 =====
6 管理系统类
7 类名： StudentManagerSystem
8 属性：
9     所有学生信息
10     学生列表 = []
11     学生字典 = {'学号': 一个学生对象, }
12 方法： 增删改查 显示菜单信息,
13
14 学生： 字典
15 类名： Student
16 属性：
17     姓名 name
```

```
18         年龄 age
19         性别 gender
20         学号 stu_id
21 方法:
22         init
23         str
```

面向对象开发的一般方式

```
1  1. 一个类作为一个 python 文件
2  2. 定义一个 main.py 作为程序代码的启动文件
3
4  =====
5  代码文件的组成:
6      student.py      # 定义 Student 类
7      student_manager_system.py  # 定义管理系统类
8      main.py        # 作为程序的启动代码
```

搭建框架代码

实现添加学生的功能

```
1  1. 使用 input 获取学生的信息
2  2. 使用学生信息, 创建学生对象
3  3. 将学生对象添加的字典中
```

```

17 def insert_student(self):
18     # 1. 使用 input 获取学生的信息
19     stu_id = input('请输入学生学号:')
20     # 代码优化, 判断学生信息是否存在, 学号是否存在. 判断字典的key是否存在
21     if stu_id in self.stu_dicts: # key存在返回True
22         print('学生信息已经存在, 不需要再次添加.....')
23         return
24     name = input('请输入学生名字:')
25     age = input('请输入学生年龄:')
26     gender = input('请输入学生性别:')
27
28     # 2. 使用学生信息, 创建学生对象 学生类(参数)
29     stu = student.Student(stu_id, name, age, gender)
30     # 3. 将学生对象添加的字典中 字典['key'] = 数据值
31     self.stu_dicts[stu_id] = stu

```

删除/修改/查询 学生

- 1 1. 使用 `input` 输入学生学号
- 2 2. 判断学生信息是否存在
- 3 3. 存在进行操作

保存

```
1  {'11': 对象, '22': 对象}
2  {'11': 0x100, '22': 0x200}
3  str({'11': 0x100, '22': 0x200})
4
5  把具体的学生信息保存到文件中
6  11, aa, 18, m
7  22, bb, 16, f
8
9  for x in {}.values():
10     pass
```

读取

1	11, aa, 18, m
2	22, bb, 16, f
3	

```
1
2  读取文件, 一行内容就是一个学生信息
3  readlines  读取所有行
4  ['11,aa,18,m\n', '22,bb,16,f\n']
5  将列表中的每一项数据转换为对象
6  Student(id, name, age, gender)
7  '11,aa,18,m\n'.strip() ==>
    '11,aa,18,m'.split() ==> ['11', 'aa', '18', 'm']
```